Experiment 1

1. Experiment Title

Demonstrate Inline, Internal, and External JavaScript; Use Console Methods; Create a Webpage Showing User Information and a Welcome Message.

2. Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Mozilla Firefox / Microsoft Edge
- HTML5
- CSS3
- JavaScript
- Windows Operating System

3. Experiment Program Code

```
<!DOCTYPE html>

<html lang="en">


<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Experiment No: 1 - JavaScript Methods Demo</title>


    <style>

        body {

            font-family: Arial, sans-serif;

            padding: 25px;

            margin: 0;

        }


        .container {

            max-width: 1000px;

            margin: 0 auto;

        }


        h1 {
```

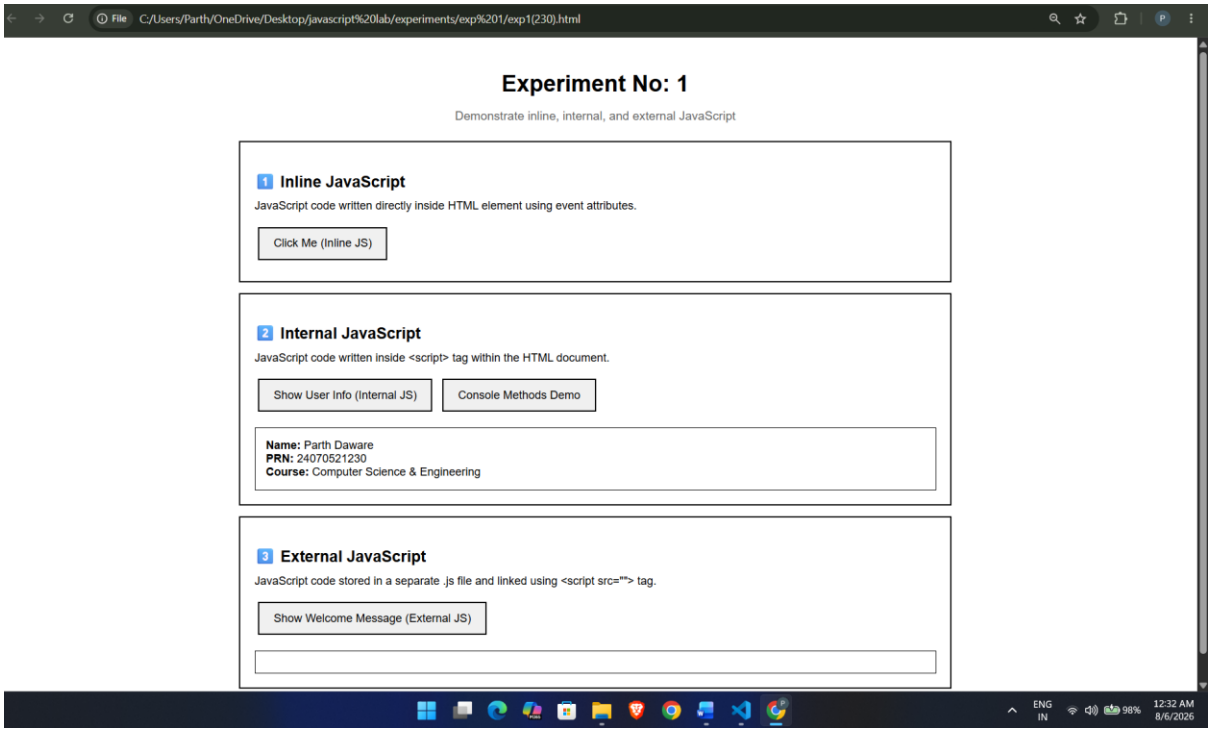
```
    text-align: center;
    font-size: 32px;
    margin-bottom: 10px;
}

.subtitle {
    text-align: center;
    color: #666;
    margin-bottom: 25px;
    font-size: 17px;
}

.section {
    padding: 20px;
    border: 2px solid black;
    margin: 15px 0;
}

.section h2 {
    font-size: 22px;
    margin-bottom: 12px;
}

.section p {
    font-size: 16px;
    margin: 10px 0;
}

button {
    padding: 12px 20px;
    border: 2px solid black;
    cursor: pointer;
    font-size: 16px;
```

```
        margin: 10px 5px;
    }

    button:hover {
        background-color: #f0f0f0;
    }

    #userInfo,
    #welcomeMessage {
        padding: 15px;
        border: 1px solid black;
        margin-top: 12px;
        font-size: 16px;
    }
</style>

<!-- Internal JavaScript -->
<script>
    console.log("=== Internal JavaScript ===");
    console.log("Internal JavaScript loaded successfully");

    // Function using Internal JavaScript
    function showUserInfo() {
        var name = "Parth Daware";
        var prn = "24070521230";
        var course = "Computer Science & Engineering";

        document.getElementById("userInfo").innerHTML =
            "<strong>Name:</strong> " + name + "<br>" +
            "<strong>PRN:</strong> " + prn + "<br>" +
            "<strong>Course:</strong> " + course;

        console.log("User Info displayed using Internal JavaScript");
```

```
            console.log("Name: " + name);
            console.log("PRN: " + prn);
        }

        // Function to demonstrate console methods
        function demonstrateConsole() {
            console.log("=== Console Methods Demo ===");
            console.log("This is console.log()");
            console.info("This is console.info()");
            console.warn("This is console.warn()");
            console.error("This is console.error()");
            alert("Check the browser console (F12) to see different console methods!");
        }
    </script>
</head>
<body>
    <div class="container">
        <h1>Experiment No: 1</h1>
        <p class="subtitle">Demonstrate inline, internal, and external JavaScript</p>
<!-- Section 1: Inline JavaScript -->
        <div class="section">
            <h2>1️⃣ Inline JavaScript</h2>
            <p>JavaScript code written directly inside HTML element using event attributes.</p>
            <button onclick="alert('Hello from Inline JavaScript!'); console.log('Inline JavaScript executed');">
                Click Me (Inline JS)
            </button>
        </div>
 <!-- Section 2: Internal JavaScript -->
        <div class="section">
            <h2>2️⃣ Internal JavaScript</h2>
<p>JavaScript code written inside &lt;script&gt; tag within the HTML document.</p>
            <button onclick="showUserInfo();">Show User Info (Internal JS)</button>
```

```
        <button onclick="demonstrateConsole();">Console Methods Demo</button>
        <div id="userInfo"></div>
      </div>
    <!-- Section 3: External JavaScript -->
      <div class="section">
        <h2>3 External JavaScript</h2>
        <p>JavaScript code stored in a separate .js file and linked using &lt;script src=""&gt; tag.</p>
        <button onclick="showWelcomeMessage();">Show Welcome Message (External JS)</button>
        <div id="welcomeMessage"></div>
      </div>
    </div>
  <!-- External JavaScript -->
    <script src="script.js"></script>
</body>
</html>
```

Output:



Case Study 1

1. Case Study Title

Create a Webpage for College Department and Link the Student Information Page with It and Write a JavaScript Program to Display "Welcome to SIT Nagpur" in an Alert Box.

2. Case Study Program Code

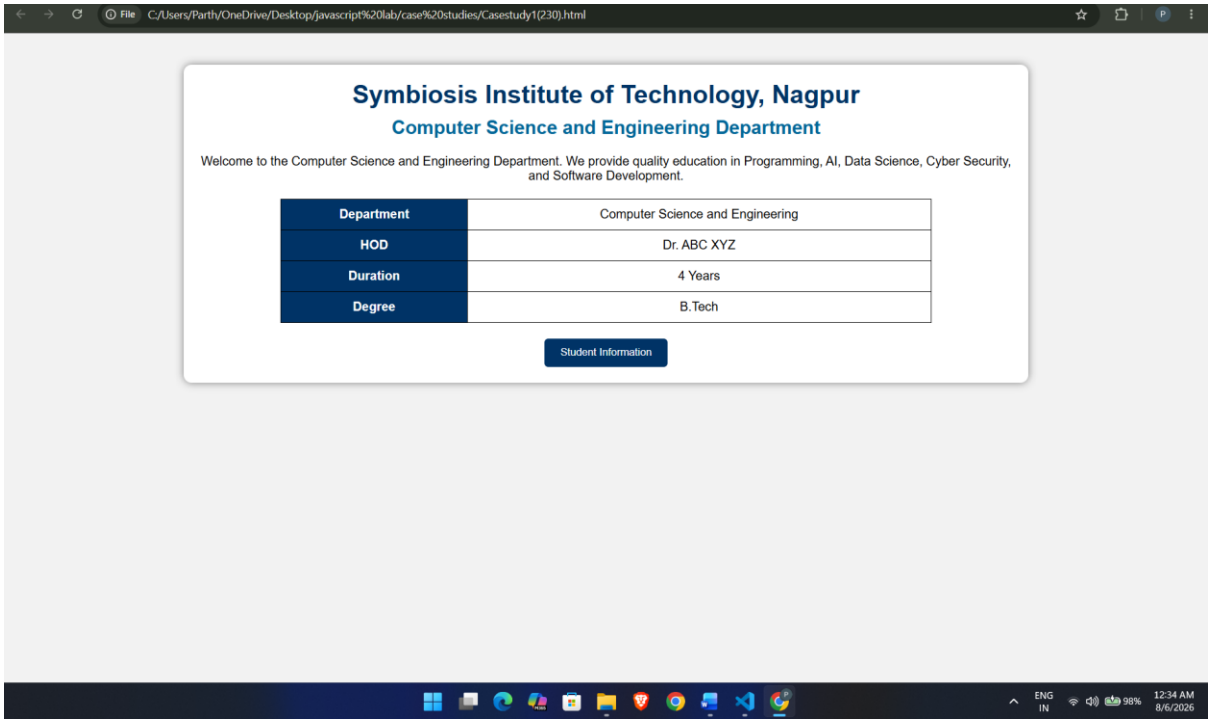
```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>College Department</title>

   <style>
      *{
         margin:0;
         padding:0;
         box-sizing:border-box;
         font-family:Arial, sans-serif;
      }
  body{
         background:#f2f2f2;
      }
    .container{
         width:70%;
         margin:40px auto;
         background:white;
         padding:20px;
         border-radius:10px;
         box-shadow:0 0 10px gray;
         text-align:center;
      }

      h1{
         color:#003366;
         margin-bottom:10px;
      }
  h2{
         color:#006699;
         margin-bottom:20px;
      }   p{   margin-bottom:20px;
```

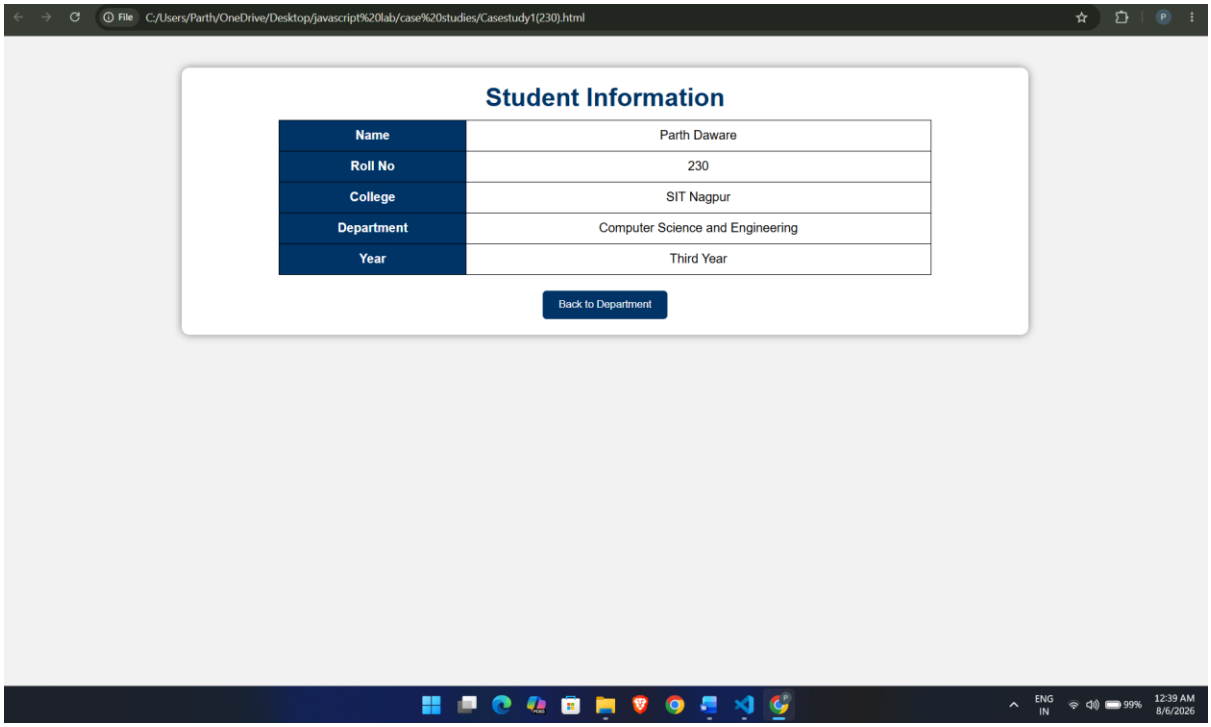
```
        }
 table{
            width:80%;
            margin:auto;
            border-collapse:collapse;
        }
 table,th,td{
            border:1px solid black;
        }     th{
            background:#003366;
            color:white;
            padding:10px;
        }    td{
            padding:10px;
        }
   button{
            margin-top:20px;
            padding:10px 20px;
            background:#003366;
            color:white;
            border:none;
            border-radius:5px;
            cursor:pointer;
        }
  button:hover{
            background:#0055aa;
        }      #studentPage{
            display:none;
        }
    </style>
</head>

<body onload="welcome()">

<!-- Department Page -->
<div class="container" id="departmentPage">
<h1>Symbiosis Institute of Technology, Nagpur</h1>
    <h2>Computer Science and Engineering Department</h2>
   <p>
        Welcome to the Computer Science and Engineering Department.
        We provide quality education in Programming, AI, Data Science,
        Cyber Security, and Software Development.</p>
<table>
```

```
        <tr>
            <th>Department</th>
            <td>Computer Science and Engineering</td>
        </tr>
    <tr>
            <th>HOD</th>
            <td>Dr. ABC XYZ</td>
        </tr>
 <tr>
            <th>Duration</th>
            <td>4 Years</td>
        </tr> <tr>
            <th>Degree</th>
            <td>B.Tech</td>
        </tr>
    </table>

    <button onclick="showStudent()">
        Student Information
    </button> </div>
<!-- Student Information Page -->
<div class="container" id="studentPage">
 <h1>Student Information</h1>
   <table>
        <tr>
            <th>Name</th>
            <td>Parth Daware</td>
        </tr>   <tr>
            <th>Roll No</th>
            <td>230</td>
        </tr>  <tr>
            <th>College</th>
            <td>SIT Nagpur</td>
        </tr>

        <tr>
            <th>Department</th>
            <td>Computer Science and Engineering</td>
        </tr>

        <tr>
            <th>Year</th>
            <td>Third Year</td>   </tr>
```

```
    </table>
    <button onclick="showDepartment()">
        Back to Department
    </button>
</div> <script>
function welcome()
{
    alert("Welcome to SIT Nagpur");
}
function showStudent()
{
    document.getElementById("departmentPage").style.display="none";
    document.getElementById("studentPage").style.display="block";
}
function showDepartment()
{
    document.getElementById("studentPage").style.display="none";
    document.getElementById("departmentPage").style.display="block";
}</script>
</body>
</html>
```

3. Output





4. Result / Conclusion

The case study was successfully implemented by creating a college department webpage and linking it to a student information page. JavaScript was used to display the welcome message **"Welcome to SIT Nagpur"** in an alert box. The webpages were designed using HTML and CSS, while JavaScript provided the required interactivity. The objectives of the case study were successfully achieved.

Experiment 2

1. Experiment Title

Use var, let, const, and demonstrate basic features like template literals and destructuring; build a billing calculator with user input**.**

2. Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Mozilla Firefox / Microsoft Edge
- HTML5
- CSS3
- JavaScript
- Windows Operating System

3. Experiment Program Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Billing Calculator (var/let/const + Features)</title>

  <style>
    body{
      font-family: Arial, Helvetica, sans-serif;
      background:#f6f7fb;
      margin:0;
      padding:28px;
      color:#111827;
    }
    .wrap{
      max-width: 920px;
      margin: 0 auto;
      display: grid;
      grid-template-columns: 1.2fr 0.8fr;
      gap: 18px;
    }
    .card{
      background:#fff;
```

```
  border:1px solid #e5e7eb;
  border-radius:14px;
  padding:18px;
  box-shadow:0 10px 24px rgba(0,0,0,0.04);
}
h1{
  margin:0 0 14px;
  font-size:20px;
}
.grid{
  display:grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}
label{
  display:block;
  font-size:13px;
  margin: 8px 0 6px;
  color:#374151;
}
input, select{
  width:100%;
  padding:10px;
  border-radius:10px;
  border:1px solid #d1d5db;
  outline:none;
  box-sizing:border-box;
  background:#fff;
}
input:focus{
  border-color:#6366f1;
  box-shadow:0 0 0 3px rgba(99,102,241,0.15);
}
button{
  margin-top:14px;
  width: 100%;
  padding: 11px 14px;
  border:0;
  border-radius:10px;
  cursor:pointer;
  font-weight:800;
  background:#2563eb;
  color:#fff;
```

```
}
button:hover{ background:#1d4ed8; }
.row{
  display:flex;
  gap:10px;
  margin-top:12px;
  flex-wrap:wrap;
}
.pill{
  padding:8px 10px;
  border-radius:999px;
  border:1px solid #e5e7eb;
  background:#fafafa;
  font-size:12px;
  color:#374151;
  user-select:none;
}
.result{
  margin-top:12px;
  padding:14px;
  border-radius:12px;
  border:1px dashed #d1d5db;
  background:#fafafa;
  white-space:pre-wrap;
  font-size:14px;
  min-height: 120px;
}
.small{
  font-size:12px;
  color:#6b7280;
  margin-top:10px;
  line-height:1.4;
}
.demoBox{
  margin-top:12px;
  padding:14px;
  border-radius:12px;
  border:1px solid #e5e7eb;
  background:#fff;
}
.demoBox h2{
  margin:0 0 10px;
  font-size:14px;
```

```
    color:#111827;
   }
   .demoCode{
    font-family: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation Mono", "Courier New", monospace;
    background:#0b1220;
    color:#e5e7eb;
    padding:12px;
    border-radius:10px;
    font-size:12px;
    white-space:pre-wrap;
   }
  </style>
</head>

<body>
  <div class="wrap">
   <div class="card">
    <h1>Billing Calculator</h1>

    <div class="grid">
     <div>
      <label for="customerName">Customer Name</label>
      <input id="customerName" type="text" placeholder="e.g., Sara" />
     </div>

     <div>
      <label for="serviceType">Service Type</label>
      <select id="serviceType">
       <option value="basic">Basic</option>
       <option value="standard">Standard</option>
       <option value="premium">Premium</option>
      </select>
     </div>

     <div>
      <label for="usage">Usage (units)</label>
      <input id="usage" type="number" min="0" step="1" value="10" />
     </div>

     <div>
      <label for="rateOverride">Custom rate (optional)</label>
```

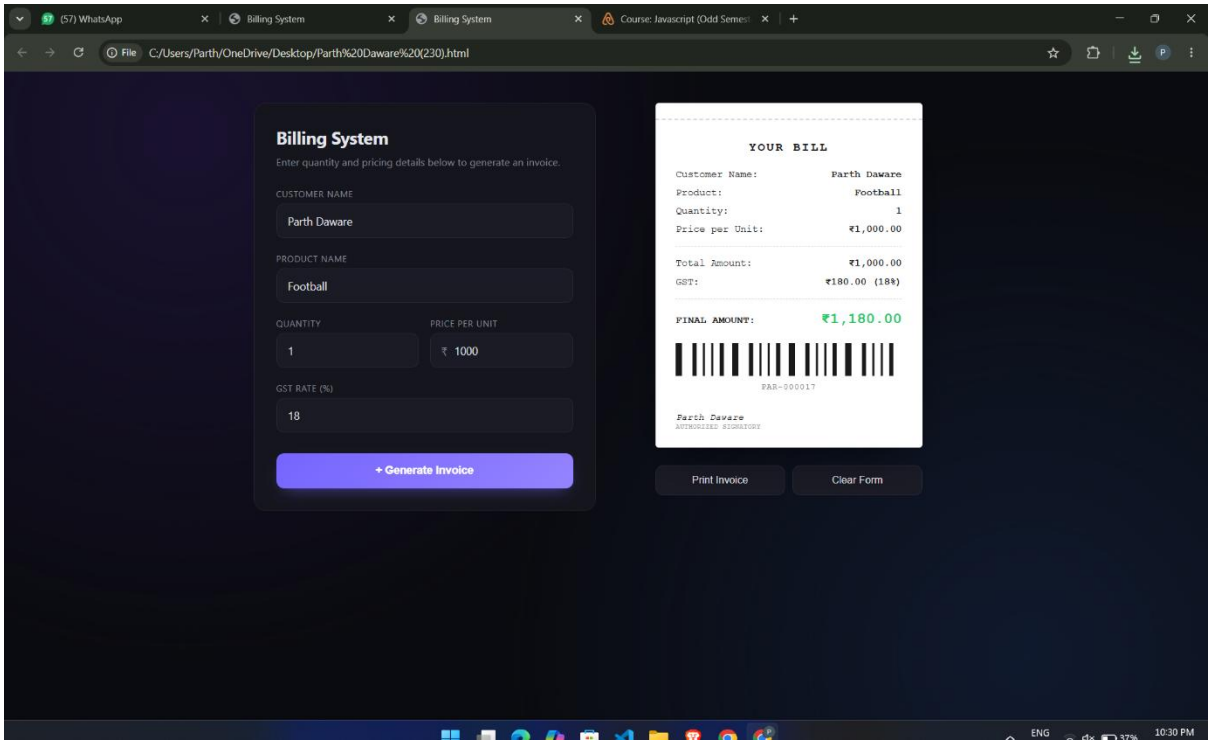
```
        <input id="rateOverride" type="number" min="0" step="0.01" placeholder="Leave blank for default" />
      </div>

      <div>
        <label for="taxRate">Tax %</label>
        <input id="taxRate" type="number" min="0" step="0.01" value="8.5" />
      </div>

      <div>
        <label for="discountRate">Discount %</label>
        <input id="discountRate" type="number" min="0" step="0.01" value="5" />
      </div>
    </div>

    <button id="calcBtn" type="button">Calculate Bill</button>

    <div class="row">
      <div class="pill">Uses var / let / const</div>
      <div class="pill">Template Literals</div>
      <div class="pill">Destructuring</div>
    </div>

    <div class="result" id="result">
      Enter details and click “Calculate Bill”.
    </div>

    <div class="small">
      Open your DevTools Console to see logs demonstrating var/let/const,
      template literals, and destructuring.
    </div>
  </div>

  <div class="card">
    <div class="demoBox">
      <h2>JS Features Demo (in console)</h2>
      <div class="demoCode" id="featurePreview"></div>
    </div>
    <div class="demoBox">
      <h2>Billing Formula</h2>
      <div class="small" style="margin-top:0">
        <div>Subtotal = usage × rate</div>
        <div>Discount = Subtotal × (discountRate / 100)</div>
```

```
        <div>AfterDiscount = Subtotal − Discount</div>
        <div>Total = AfterDiscount × (1 + taxRate / 100)</div>
       </div>
      </div>
     </div>
    </div>   <script>
     const $ = (id) => document.getElementById(id);  const serviceRates = {
      basic: 12.5,
      standard: 20,
      premium: 35
     }; function money(n) {
      return new Intl.NumberFormat(undefined, {
       style: "currency",
       currency: "USD"
      }).format(n);
     } function parseNumOrNull(val) {
      const trimmed = String(val ?? "").trim();
      if (!trimmed) return null;
      const n = Number(trimmed);
      return Number.isFinite(n) ? n : null;
     }  $("featurePreview").textContent = [
      "var: function-scoped value (demo only)",
      "let: block-scoped variable (demo)",
      "const: block-scoped constant (demo)",
      "Template literals: `Welcome, ${name}`",
      "Destructuring: const {basic, standard} = serviceRates"
     ].join("\n");
    $("calcBtn").addEventListener("click", () => {
      var debugVar = "var-demo";
      let debugLet = "let-demo";
      const debugConst = "const-demo";

      console.log("---- var/let/const demo ----");
      console.log("debugVar (var):", debugVar);
      console.log("debugLet (let):", debugLet);
      console.log("debugConst (const):", debugConst);
    debugLet = "let-demo (updated)";
      console.log("debugLet updated:", debugLet);
    console.log("---- Destructuring demo ----");
      const { basic, standard, premium } = serviceRates;
      console.table([{ basic, standard, premium }]);
     console.log("---- Template literal demo ----");
      const name = $("customerName").value.trim() || "Customer";
```

```
    console.log(`Welcome, ${name}! Now calculating your bill...`);

    const serviceType = $("serviceType").value;
    const usage = Number($("usage").value);
    const taxRate = Number($("taxRate").value);
    const discountRate = Number($("discountRate").value);
    const customRate = parseNumOrNull($("rateOverride").value);
const rate = (customRate ?? serviceRates[serviceType]);

    if (!Number.isFinite(usage) || usage < 0) {
      console.error("Invalid usage.");
      $("result").textContent = "Please enter a valid usage (0 or more).";
      return;
    } if (!Number.isFinite(taxRate) || taxRate < 0) {
      console.error("Invalid tax rate.");
      $("result").textContent = "Please enter a valid tax % (0 or more).";
      return;
    } if (!Number.isFinite(discountRate) || discountRate < 0) {
      console.error("Invalid discount rate.");
      $("result").textContent = "Please enter a valid discount % (0 or more).";
      return;
    }  const subtotal = usage * rate;
    const discount = subtotal * (discountRate / 100);
    const afterDiscount = subtotal - discount;
    const tax = afterDiscount * (taxRate / 100);
    const total = afterDiscount + tax;

    console.group("---- Calculation logs ----");
    console.log("serviceType:", serviceType);
    console.log("rate:", rate);
    console.log("usage:", usage);
    console.log("subtotal:", subtotal);
    console.log("discount:", discount);
    console.log("afterDiscount:", afterDiscount);
    console.log("tax:", tax);
    console.log("total:", total);
    console.groupEnd();

    const userSummary = { name, serviceType, usage, rate, taxRate, discountRate };
    console.table([userSummary]);

    const message = `
Customer: ${name}
```

```
Service: ${serviceType.charAt(0).toUpperCase() + serviceType.slice(1)}
Usage: ${usage}
Rate: ${money(rate)}

Subtotal: ${money(subtotal)}
Discount (${discountRate}%): -${money(discount)}
After Discount: ${money(afterDiscount)}
Tax (${taxRate}%): +${money(tax)}

Total Due: ${money(total)}
    `.trim();

    $("result").textContent = message;

    console.log("Done. Total Due rendered.");
    console.log(`var-demo value still accessible: ${debugVar}`);
  });
 </script>
</body>
</html>
```

Output:



Case Study 2

1. Case Study Title

Develop the webpage for an online shopping website with using HTML CSS and HTML5 tags and calculate the bill.

2. Case Study Program Code

The case study consists of the following files:
• **index.html** – Displays the Premium Bill Calculator webpage containing a form to input customer and product details for generating an invoice.

• **script.js** – Processes the form data, performs calculations (like base amount, discount, GST, and final total) by demonstrating various data types and type coercions, and dynamically updates the invoice summary on the webpage.

• **style.css** – Contains the styling rules for the layout, typography, and visual elements of the bill calculator application.

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Online Shopping Website</title>

<style>
body{
   font-family: Arial, sans-serif;
   background:#f4f4f4;
   margin:0;
}

header{
   background:#0077cc;
   color:white;
   text-align:center;
```

```
    padding:20px;
}

nav{
    background:#333;
    padding:10px;
    text-align:center;
}

nav a{
    color:white;
    text-decoration:none;
    margin:15px;
}

section{
    width:80%;
    margin:20px auto;
    background:white;
    padding:20px;
    border-radius:10px;
    box-shadow:0px 0px 10px gray;
}

table{
    width:100%;
    border-collapse:collapse;
}

table,th,td{
    border:1px solid black;
}

th{
    background:#0077cc;
    color:white;
    padding:10px;
```

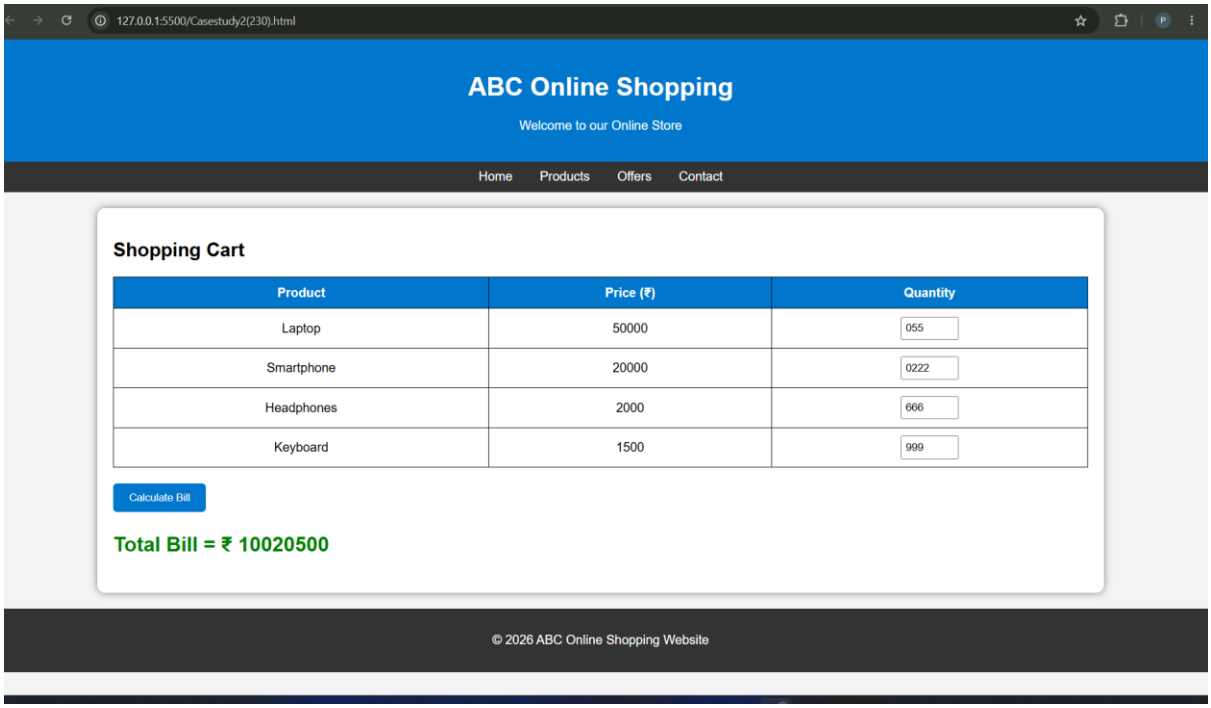
```
}

td{
    padding:10px;
    text-align:center;
}

input{
    width:60px;
    padding:5px;
}

button{
    margin-top:20px;
    padding:10px 20px;
    background:#0077cc;
    color:white;
    border:none;
    border-radius:5px;
    cursor:pointer;
}

button:hover{
    background:#005fa3;
}

footer{
    background:#333;
    color:white;
    text-align:center;
    padding:15px;
    margin-top:20px;
}

#result{
    margin-top:20px;
    font-size:22px;
```

```
    color:green;
    font-weight:bold;
}
</style>
</head>

<body>

<header>
<h1>ABC Online Shopping</h1>
<p>Welcome to our Online Store</p>
</header>

<nav>
<a href="#">Home</a>
<a href="#">Products</a>
<a href="#">Offers</a>
<a href="#">Contact</a>
</nav>

<section>

<h2>Shopping Cart</h2>

<table>
<tr>
<th>Product</th>
<th>Price (₹)</th>
<th>Quantity</th>
</tr>

<tr>
<td>Laptop</td>
<td>50000</td>
<td><input type="number" id="q1" value="0" min="0"></td>
</tr>
```

```
<tr>
<td>Smartphone</td>
<td>20000</td>
<td><input type="number" id="q2" value="0" min="0"></td>
</tr>

<tr>
<td>Headphones</td>
<td>2000</td>
<td><input type="number" id="q3" value="0" min="0"></td>
</tr>

<tr>
<td>Keyboard</td>
<td>1500</td>
<td><input type="number" id="q4" value="0" min="0"></td>
</tr>

</table>

<button onclick="calculateBill()">Calculate Bill</button>

<div id="result"></div>

</section>

<footer>
<p>&copy; 2026 ABC Online Shopping Website</p>
</footer>

<script>

function calculateBill(){

let laptop = 50000 * Number(document.getElementById("q1").value);
let mobile = 20000 * Number(document.getElementById("q2").value);
let headphone = 2000 * Number(document.getElementById("q3").value);
```

```
let keyboard = 1500 * Number(document.getElementById("q4").value);

let total = laptop + mobile + headphone + keyboard;

document.getElementById("result").innerHTML =
"<h3>Total Bill = ₹ " + total + "</h3>";

}

</script>

</body>
</html>
```

Output:



Result/Conclusion:

Hence, we have successfully developed the Premium Bill Calculator webpage. Through this case study, we practically learned how to declare variables using var, let, and const, utilize various primitive data types (such as strings, numbers, and booleans), and perform type coercion and explicit type conversions in JavaScript to compute the final bill amount and dynamically display the generated invoice.

Experiment 3

1. Experiment Title

Implement control structures and form validation; create a grading system based on user-entered marks.

2. Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Mozilla Firefox / Microsoft Edge
- HTML5
- CSS3
- JavaScript
- Windows Operating System

3. Experiment Program Code

```
<!DOCTYPE html>

<html lang="en">


<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Experiment No: 1 - JavaScript Methods Demo</title>


    <style>

        body {

            font-family: Arial, sans-serif;

            padding: 25px;

            margin: 0;

        }
```

```
.container {
    max-width: 1000px;
    margin: 0 auto;
}

h1 {
    text-align: center;
    font-size: 32px;
    margin-bottom: 10px;
}

.subtitle {
    text-align: center;
    color: #666;
    margin-bottom: 25px;
    font-size: 17px;
}

.section {
    padding: 20px;
    border: 2px solid black;
    margin: 15px 0;
}

.section h2 {
    font-size: 22px;
    margin-bottom: 12px;
}
```

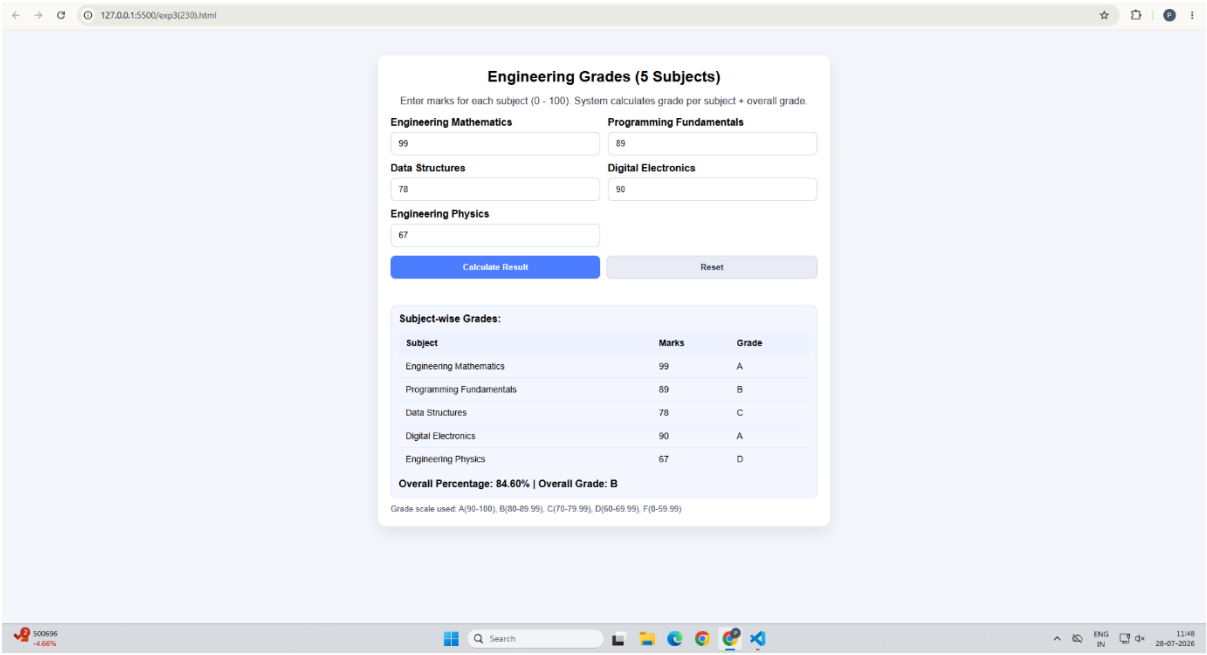
```
    .section p {
        font-size: 16px;
        margin: 10px 0;
    }

    button {
        padding: 12px 20px;
        border: 2px solid black;
        cursor: pointer;
        font-size: 16px;
        margin: 10px 5px;
    }

    button:hover {
        background-color: #f0f0f0;
    }

    #userInfo,
    #welcomeMessage {
        padding: 15px;
        border: 1px solid black;
        margin-top: 12px;
        font-size: 16px;
    }
</style>

<!-- Internal JavaScript -->
<script>
    console.log("=== Internal JavaScript ===");
```

```
        console.log("Internal JavaScript loaded successfully");

        // Function using Internal JavaScript
        function showUserInfo() {
            var name = "Parth Daware";
            var prn = "24070521230";
            var course = "Computer Science & Engineering";

            document.getElementById("userInfo").innerHTML =
                "<strong>Name:</strong> " + name + "<br>" +
                "<strong>PRN:</strong> " + prn + "<br>" +
                "<strong>Course:</strong> " + course;

            console.log("User Info displayed using Internal JavaScript");
            console.log("Name: " + name);
            console.log("PRN: " + prn);
        }

        // Function to demonstrate console methods
        function demonstrateConsole() {
            console.log("=== Console Methods Demo ===");
            console.log("This is console.log()");
            console.info("This is console.info()");
            console.warn("This is console.warn()");
            console.error("This is console.error()");
            alert("Check the browser console (F12) to see different console methods!");
        }
    </script>
</head>
```

```
<body>
  <div class="container">
    <h1>Experiment No: 1</h1>
    <p class="subtitle">Demonstrate inline, internal, and external JavaScript</p>

    <!-- Section 1: Inline JavaScript -->
    <div class="section">
      <h2>1 Inline JavaScript</h2>
      <p>JavaScript code written directly inside HTML element using event attributes.</p>
      <button onclick="alert('Hello from Inline JavaScript!'); console.log('Inline JavaScript executed');">
        Click Me (Inline JS)
      </button>
    </div>

    <!-- Section 2: Internal JavaScript -->
    <div class="section">
      <h2>2 Internal JavaScript</h2>
      <p>JavaScript code written inside &lt;script&gt; tag within the HTML document.</p>
      <button onclick="showUserInfo();">Show User Info (Internal JS)</button>
      <button onclick="demonstrateConsole();">Console Methods Demo</button>
      <div id="userInfo"></div>
    </div>

    <!-- Section 3: External JavaScript -->
    <div class="section">
      <h2>3 External JavaScript</h2>
```

```
        <p>JavaScript code stored in a separate .js file and linked using &lt;script src=""&gt; tag.</p>

        <button onclick="showWelcomeMessage();">Show Welcome Message (External JS)</button>

        <div id="welcomeMessage"></div>

    </div>

</div>


<!-- External JavaScript -->

<script src="script.js"></script>

</body>


</html>
```

Output:



Case Study 3

1. Case Study Title

Write a Javascript program that accepts students marks and displays the scholarship category.

2. Case Study Program Code

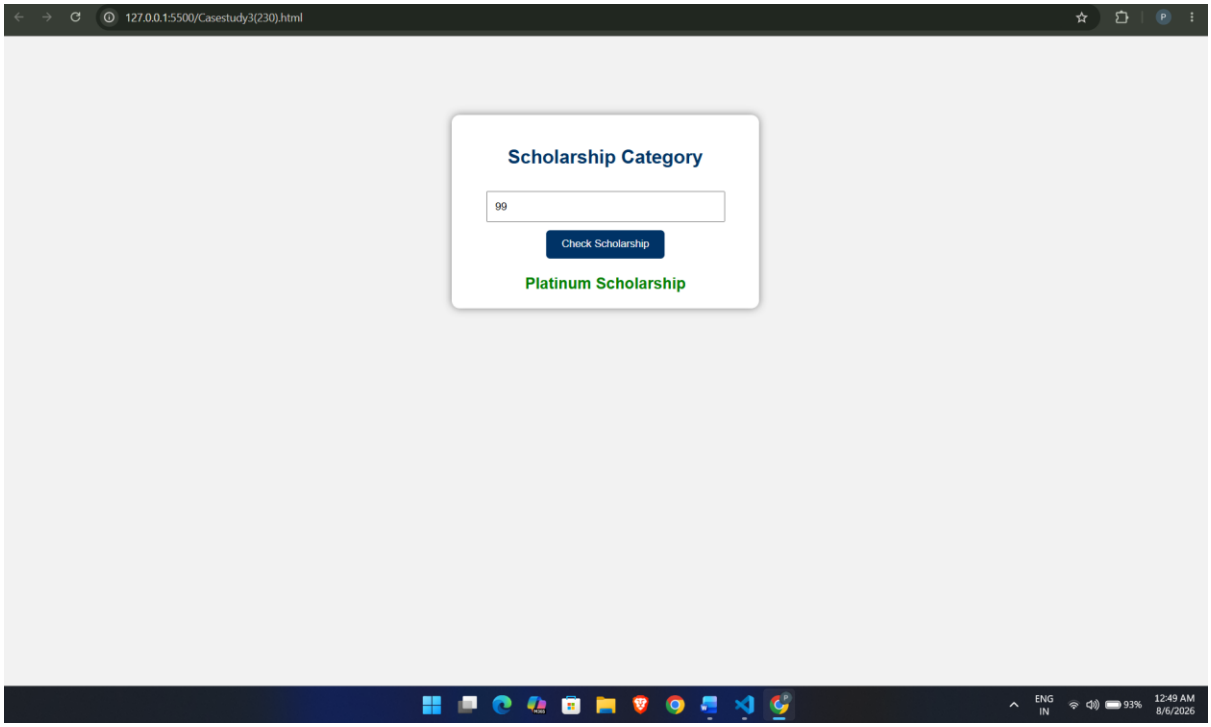
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Scholarship Category</title>

<style>
body{
    font-family: Arial, sans-serif;
    background:#f2f2f2;
}

.container{
    width:350px;
    margin:100px auto;
    background:white;
    padding:20px;
    text-align:center;
    border-radius:10px;
    box-shadow:0 0 10px gray;
}
```

```
h2{
    color:#003366;
}

input{
    width:80%;
    padding:10px;
    margin:10px 0;
}

button{
    padding:10px 20px;
    background:#003366;
    color:white;
    border:none;
    border-radius:5px;
    cursor:pointer;
}

button:hover{
    background:#0055aa;
}

#result{
    margin-top:20px;
    font-size:20px;
    font-weight:bold;
    color:green;
}
```

```
</style>

</head>
<body>

<div class="container">

<h2>Scholarship Category</h2>

<input type="number" id="marks" placeholder="Enter Student Marks">

<br>

<button onclick="checkScholarship()">Check Scholarship</button>

<div id="result"></div>

</div>

<script>

function checkScholarship(){

    let marks = Number(document.getElementById("marks").value);
    let category="";

    if(marks<0 || marks>100){
        category="Invalid Marks";
    }
```

```
    else if(marks>=90){
        category="Platinum Scholarship";
    }
    else if(marks>=75){
        category="Gold Scholarship";
    }
    else if(marks>=60){
        category="Silver Scholarship";
    }
    else if(marks>=40){
        category="Bronze Scholarship";
    }
    else{
        category="No Scholarship";
    }

    document.getElementById("result").innerHTML=category;

}

</script>

</body>
</html>
      checkScholarship();
   }
});
```

Output:



Result / Conclusion:

Hence, we have successfully developed the Scholarship Eligibility Checker webpage. Through this case study, we practically learned how to implement control structures, specifically if-else conditional statements in JavaScript, to evaluate user input, make logical decisions, and dynamically update the webpage's UI to display the appropriate scholarship category based on the provided marks.

Experiment 4

1. Experiment Title

Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required

- Visual Studio Code (VS Code)
- Google Chrome / Mozilla Firefox / Microsoft Edge
- HTML5
- CSS3
- JavaScript
- Windows Operating System

3. Experiment Program Code

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Palindrome Checker (Functions + Scope + Closures + Try/Catch)</title>


  <style>

    body {

      font-family: Arial, Helvetica, sans-serif;

      background: #f6f7fb;

      margin: 0;

      padding: 28px;

      color: #111827;

    }

    .wrap {

      max-width: 720px;

      margin: 0 auto;
```

```
}
.card {
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 14px;
  padding: 18px;
  box-shadow: 0 10px 24px rgba(0,0,0,0.04);
}
h1 { margin: 0 0 12px; font-size: 20px; }
label {
  display: block;
  font-size: 13px;
  margin: 10px 0 6px;
  color: #374151;
  font-weight: 700;
}
input {
  width: 100%;
  padding: 10px;
  border-radius: 10px;
  border: 1px solid #d1d5db;
  outline: none;
  box-sizing: border-box;
  background: #fff;
}
input:focus {
  border-color: #6366f1;
  box-shadow: 0 0 0 3px rgba(99,102,241,0.15);
}
```

```
.row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  margin-top: 12px;
}
.actions {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin-top: 14px;
  align-items: center;
}
button {
  padding: 11px 14px;
  border: 0;
  border-radius: 10px;
  cursor: pointer;
  font-weight: 800;
  background: #2563eb;
  color: #fff;
  flex: 1;
  min-width: 180px;
}
button.secondary { background: #111827; }
button:hover { filter: brightness(0.95); }
.msg {
  margin-top: 14px;
  padding: 14px;
```

```
      border-radius: 12px;
      border: 1px dashed #d1d5db;
      background: #fafafa;
      min-height: 96px;
      white-space: pre-wrap;
      font-size: 14px;
      line-height: 1.4;
    }
    .small {
      margin-top: 10px;
      font-size: 12px;
      color: #6b7280;
      line-height: 1.4;
    }
    @media (max-width: 640px) {
      .row { grid-template-columns: 1fr; }
      button { min-width: 100%; }
    }
  </style>
</head>

<body>
  <div class="wrap">
    <div class="card">
      <h1>Palindrome Checker</h1>

      <label for="text">Enter text</label>
      <input id="text" type="text" placeholder="e.g., racecar / A man, a plan, a canal: Panama"
/>
```

```
    <div class="row">
      <div>
        <label for="mode">Normalization mode</label>
        <input id="mode" type="text" value="ignore-nonletters" />
        <div class="small">Type: ignore-nonletters (default) or strict</div>
      </div>
      <div>
        <label for="minLen">Min length (validation)</label>
        <input id="minLen" type="number" min="0" step="1" value="1" />
        <div class="small">If input is shorter than this, it throws and is caught.</div>
      </div>
    </div>

    <div class="actions">
      <button id="checkBtn" type="button">Check Palindrome</button>
      <button class="secondary" id="resetBtn" type="button">Reset</button>
    </div>

    <div class="msg" id="output">Enter text and click “Check Palindrome”.</div>

    <div class="small">
      Open DevTools console to see logs for function types, scope, closures, and try/catch.
    </div>
  </div>
</div>

<script>
  const $ = (id) => document.getElementById(id);
```

```
const outputEl = $("output");

function setOutput(text, ok) {
  outputEl.textContent = text;
  outputEl.style.borderColor = ok ? "#86efac" : "#fca5a5";
  outputEl.style.background = ok ? "#ecfdf5" : "#fef2f2";
}

function isPalindromeNormalized(normalized) {
  const s = normalized;
  let left = 0;
  let right = s.length - 1;

  while (left < right) {
    if (s[left] !== s[right]) return false;
    left++;
    right--;
  }
  return true;
}

const normalize = (input, mode) => {
  if (mode === "strict") return input;
  const lower = input.toLowerCase();
  return lower.replace(/[^a-z0-9]/g, "");
};

function createChecker(mode) {
  const capturedMode = mode;
```

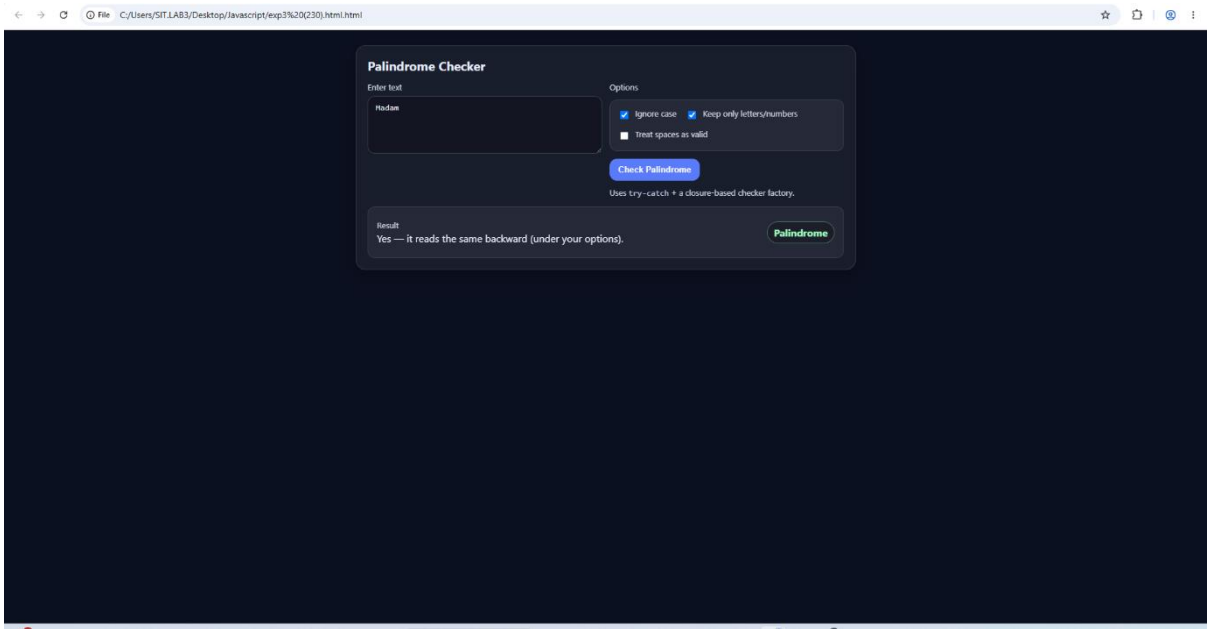
```
  function checker(text) {
    const normalized = normalize(text, capturedMode);
    const result = isPalindromeNormalized(normalized);

    return { normalized, result, mode: capturedMode };
  }

  return checker;
}

(function demoFunctionTypesScopeClosures() {
  console.log("---- Function Types + Scope + Closures demo ----");

  var scopeVar = "var is function-scoped";
  let scopeLet = "let is block-scoped";
  const scopeConst = "const is block-scoped constant";

  console.log("scopeVar:", scopeVar);
  console.log("scopeLet:", scopeLet);
  console.log("scopeConst:", scopeConst);

  if (true) {
    let blockLet = "inside block (let)";
    const blockConst = "inside block (const)";
    console.log("blockLet:", blockLet);
    console.log("blockConst:", blockConst);
  }
```

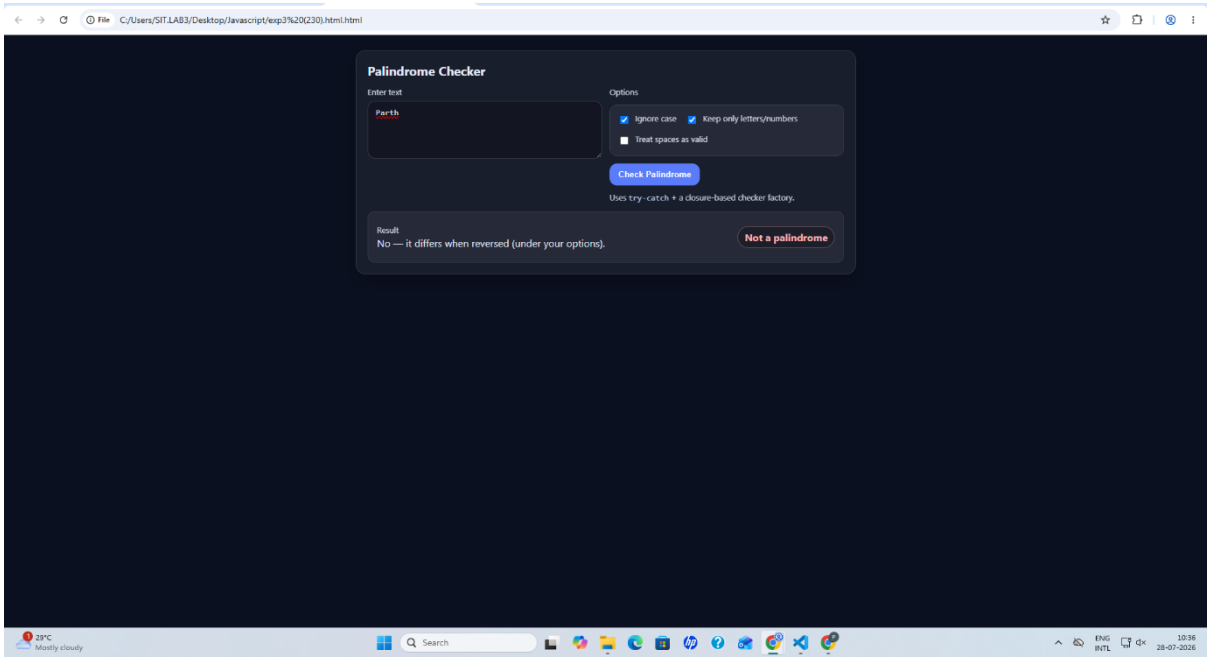
```
  const defaultMode = "ignore-nonletters";
  const checker = createChecker(defaultMode);

  const closureExample = checker("racecar");
  console.log("Closure capturedMode:", closureExample.mode);
  console.log("Closure normalized:", closureExample.normalized);
  console.log("Closure result:", closureExample.result);

  console.log("---- End demo ----");
})();

$("checkBtn").addEventListener("click", () => {
  try {
    const rawText = $("text").value;
    const minLen = Number($("minLen").value);
    const mode = ($("mode").value || "ignore-nonletters").trim();

    if (mode !== "ignore-nonletters" && mode !== "strict") {
      throw new Error("Mode must be 'ignore-nonletters' or 'strict'.");
    }

    if (!Number.isFinite(minLen) || minLen < 0) {
      throw new Error("Min length must be a valid number (>= 0).");
    }

    if ((rawText ?? "").trim().length < minLen) {
      throw new Error(`Input is shorter than min length (${minLen}).`);
    }
```

```
    const checker = createChecker(mode);
    const { normalized, result } = checker(rawText);

    const message =
      `Input: ${rawText}\n` +
      `Mode: ${mode}\n` +
      `Normalized: ${normalized}\n\n` +
      (result
        ? "✅ It is a palindrome."
        : "❌ It is NOT a palindrome.");

    console.log("Try block success:", { rawText, mode, minLen, normalized, result });
    setOutput(message, result);
  } catch (err) {
    console.error("Caught error:", err);
    setOutput(`⚠️ Error: ${err.message}`, false);
  }
});

$("resetBtn").addEventListener("click", () => {
  $("text").value = "";
  $("mode").value = "ignore-nonletters";
  $("minLen").value = 1;
  setOutput("Enter text and click “Check Palindrome”.", true);
  $("text").focus();
});

$("text").addEventListener("keydown", (e) => {
  if (e.key === "Enter") $("checkBtn").click();
```

```
  });
 </script>
</body>
</html>
```

Output:





Case Study 4

1. Case Study Title

Real-Time Case Study: ATM Card PIN Verification Using JavaScript Functions to verify whether a customer's PIN is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope and closure concepts.

2. Case Study Program Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ATM PIN Verification</title>

<style>
body{
font-family: Arial, sans-serif;
background:#f2f2f2;
}

.container{
width:400px;
margin:80px auto;
background:white;
padding:20px;
```

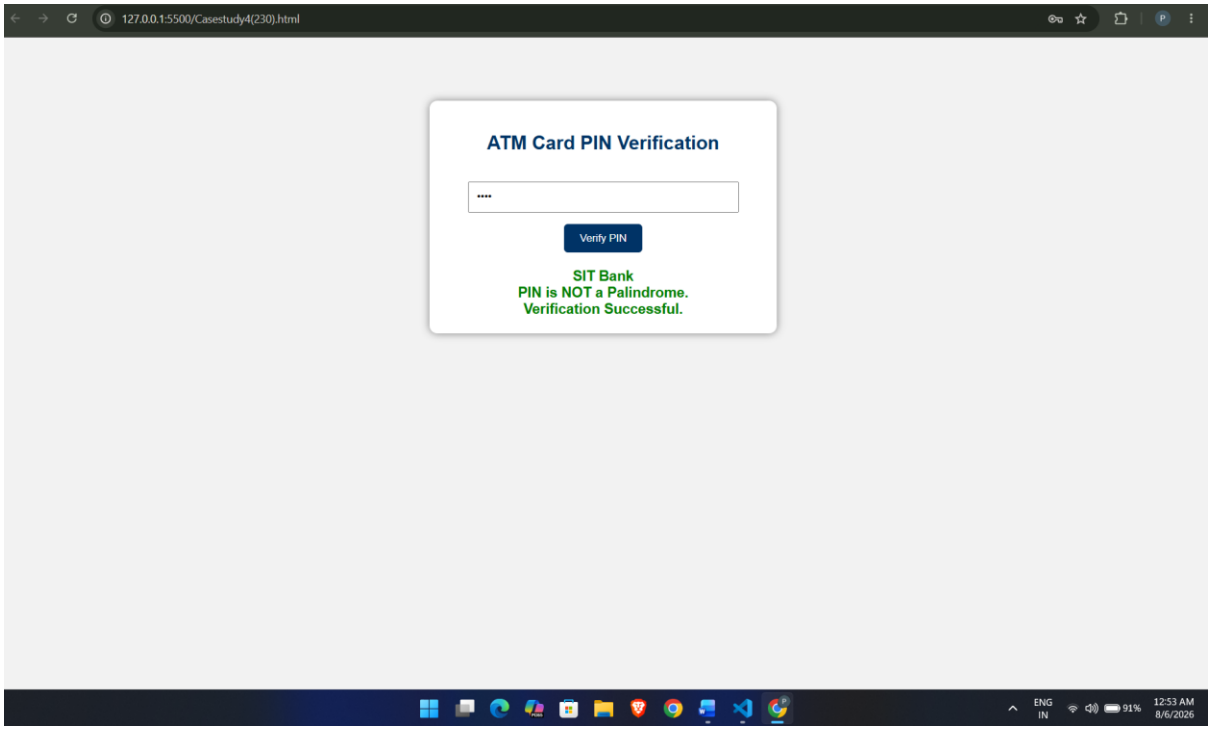
```
border-radius:10px;
text-align:center;
box-shadow:0 0 10px gray;
}

h2{
color:#003366;
}

input{
width:80%;
padding:10px;
margin:15px;
}

button{
padding:10px 20px;
background:#003366;
color:white;
border:none;
border-radius:5px;
cursor:pointer;
}

button:hover{
background:#0055aa;
}

#result{
```

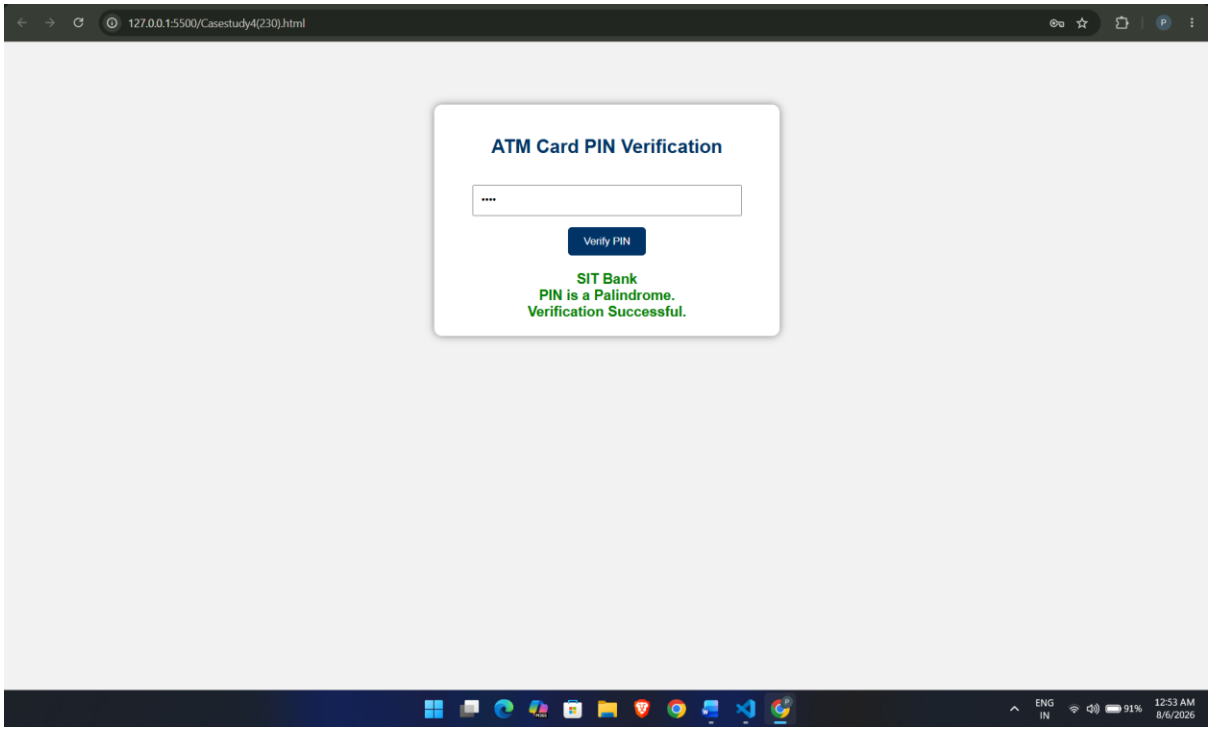
```
margin-top:20px;
font-size:18px;
font-weight:bold;
color:green;
}
</style>
</head>

<body>

<div class="container">

<h2>ATM Card PIN Verification</h2>

<input type="password" id="pin" placeholder="Enter 4-digit PIN">

<br>

<button onclick="verifyPIN()">Verify PIN</button>

<div id="result"></div>

</div>

<script>

// Function Declaration
function isPalindrome(pin){
let reverse = pin.split("").reverse().join("");
```

```
return pin === reverse;
}

// Function Expression
const validatePIN = function(pin){
return /^\d{4}$/.test(pin);
};

// Arrow Function
const displayResult = (message) => {
document.getElementById("result").innerHTML = message;
};

// Scope Example
function verifyPIN(){

let pin = document.getElementById("pin").value;

if(!validatePIN(pin)){
displayResult("Enter a valid 4-digit PIN.");
return;
}

// Closure Example
function securityCheck(){
let bankName = "SIT Bank";
return function(){
if(isPalindrome(pin)){
displayResult(bankName + "<br>PIN is a Palindrome.<br>Verification Successful.");
```

```
}else{
displayResult(bankName + "<br>PIN is NOT a Palindrome.<br>Verification Successful.");
}
};
}let check = securityCheck();
check();
}
</script>
</body>
</html>
```

Output:





Result/Conclusion:

Case Study 4 successfully demonstrates how to use closures in JavaScript to securely maintain private data (like tracking failed PIN attempts) without exposing it to the global scope. It provides a functional ATM interface that validates palindromic PINs and securely blocks the user after 3 invalid attempts.